



Course Code : DSC101
Course Name : Introduction to Data Science
Lecturer : Mr Toa Chean Khim
Academic Session : 2023/09
Assessment Title : Assignment
Submission Due Date : 25/12/2023

Prepared by :	Student ID	Student Name
	DSC2302025	SEAN WONG WEE KAI
	DSC2304093	BONG XUAN JING

Date Received : _____

Feedback from Lecturer:	Mark:
-------------------------	-------

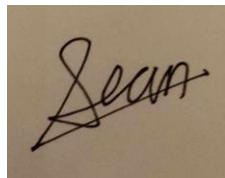
Own Work Declaration

I/We hereby understand my/our work would be checked for plagiarism or other misconduct, and the softcopy would be saved for future comparison(s).

I/We hereby confirm that all the references or sources of citations have been correctly listed or presented and I/we clearly understand the serious consequence caused by any intentional or unintentional misconduct.

In addition, I/we affirm that this submission does not contain any materials generated by AI tools, including direct copying and pasting of text or paraphrasing. This work is my/our original creation, and it has not been based on any work of other students (past or present), nor has it been previously submitted to any other course or institution.

Signature:

A handwritten signature in black ink on a light brown background. The signature appears to be "Sean" written in a cursive, stylized font.A handwritten signature in black ink on a light grey background. The signature appears to be "Conlin" written in a cursive, stylized font.

Date: 25/12/2023

1.0 Data Collection

The data was collected from Coffee Bean Sales Raw Dataset, which was retrieved from kaggle.com under the name of Saad Haroon. An additional file was downloaded from <https://simplemaps.com/data/us-zips> for assistance in further classifying the data in the Coffee Bean Sales Raw Dataset, which will be mentioned in the “Data Preprocessing” section.

2.0 Data Preprocessing

The dataset had three separate sheets which were “Customers”, “Orders”, and “Products”. The data retrieved was in the form of xlsx file, so when we imported the file, we had to change it to csv form for more efficient data preprocessing.

```
import pandas as pd

# Read and store content of an Excel file
read_file_orders = pd.read_excel("D:/2023_Seal/XMUM/2023/datascienceassignment.xlsx", sheet_name='orders')
read_file_customers = pd.read_excel("D:/2023_Seal/XMUM/2023/datascienceassignment.xlsx", sheet_name='customers')
read_file_products = pd.read_excel("D:/2023_Seal/XMUM/2023/datascienceassignment.xlsx", sheet_name='products')

# Write each dataframe object into a separate CSV file
read_file_orders.to_csv("D:/2023_Seal/XMUM/2023/orders.csv", index=None, header=True)
read_file_customers.to_csv("D:/2023_Seal/XMUM/2023/customers.csv", index=None, header=True)
read_file_products.to_csv("D:/2023_Seal/XMUM/2023/products.csv", index=None, header=True)

# Read CSV files and convert into dataframe objects
df_orders = pd.read_csv("D:/2023_Seal/XMUM/2023/orders.csv")
df_customers = pd.read_csv("D:/2023_Seal/XMUM/2023/customers.csv")
df_products = pd.read_csv("D:/2023_Seal/XMUM/2023/products.csv")

# Printing the dataframes
print("Orders DataFrame:")
print(df_orders)

print("\nCustomers DataFrame:")
print(df_customers)

print("\nProducts DataFrame:")
print(df_products)
```

After that, we printed out the three sheets and deleted empty columns in “Orders”.

```
# Removing empty columns in orders dataframe
columns_to_drop_orders = ['Customer Name', 'Email', 'Country', 'Coffee Type', 'Roast Type', 'Size', 'Unit Price', 'Sales']
read_file_orders.drop(columns=columns_to_drop_orders, inplace=True)
read_file_orders.to_csv("D:/2023_Seal/XMUM/2023/orders.csv", index=None, header=True)
df_orders = pd.read_csv("D:/2023_Seal/XMUM/2023/orders.csv")
print("Orders DataFrame after slight modification:")
print(df_orders)
```

```
Orders DataFrame after slight modification:
   Order ID  Order Date  Customer ID  Product ID  Quantity
0  QEV-37451-860  2019-09-05  17670-51384-MA  R-M-1         2
1  QEV-37451-860  2019-09-05  17670-51384-MA  E-M-0.5       5
2  FAA-43335-268  2021-06-17  21125-22134-PX  A-L-1         1
3  KAC-83089-793  2021-07-15  23806-46781-OU  E-M-1         2
4  KAC-83089-793  2021-07-15  23806-46781-OU  R-L-2.5       2
..      ...      ...      ...      ...      ...
995  RLM-96511-467  2020-01-06  43014-53743-XK  R-L-2.5       1
996  AEZ-13242-456  2022-05-04  62494-09113-RP  R-M-0.5       5
997  UME-75640-698  2019-10-11  62494-09113-RP  A-M-0.5       4
998  GJC-66474-557  2021-08-03  64965-78386-MY  A-D-1         1
999  IRV-20769-219  2020-10-15  77131-58092-GE  E-M-0.2       3
```

```
[1000 rows x 5 columns]
```

Next, we merged the three sheets together into one data frame so we can conduct our analysis later.

```
# Merging customers sheet and orders sheet into one dataframe
data_customer_order=pd.merge(df_customers,df_orders ,on='Customer ID')
print(data_customer_order)

data=pd.merge(df_products,data_customer_order ,on='Product ID')
print(data)
```

We did another round of checking for empty values, and we found out that all the empty values were located in the “Phone Number” and “Email Address” column. Those two columns are not needed for our objectives, so we dropped these columns along with “Customer Name” and “Address Line 1” to ensure privacy of the customers.

```
# Check for NaN values by row
nan_values_by_row = data.isna().any(axis=1)

# Filter rows with NaN values
rows_with_nan = data[nan_values_by_row]

# Print the result
print("Rows with NaN values:")
print(rows_with_nan)

# There are 317 rows that contain empty values, after observation all from email or phone number, which does not serve much purpose to data analysis, so we will omit it
# We do not need "Customer Name" and "Address Line 1" as well, so we will drop that too.
columns_to_drop = [ 'Email','Phone Number','Customer Name','Address Line 1']
data.drop(columns=columns_to_drop, inplace=True)
print(data)
```

Then, we imported another file uszips which was already in csv form so no modification was needed.

```
read_file_city = pd.read_excel("Downloads/uszips.xlsx")
read_file_city.to_csv("Downloads/uszips.csv", index=None, header=True)
df_city = pd.read_csv("Downloads/uszips.csv")
print(df_city)
```

Aside from columns “zip” and “state_name”, all other columns were dropped as they were not needed.

```
columns_to_drop = [ 'lat','lng','city','state_id','zcta','parent_zcta','population','density','county_fips','county_name','county_weights','county_names_all','county_fips_
df_city.drop(columns=columns_to_drop, inplace=True)
print(df_city)
```

The main goal of importing this file is to classify the cities in the “United States” category under the “Country” column into states. The values of “zip” were grouped together according to their respective states and later renamed to “zip_range”.

```
# Group by state_name and aggregate zip codes into a range
grouped_data = df_city.groupby('state_name')['zip'].agg(['zip_range', lambda x: f"{x.min()}-{x.max()}"]).reset_index()

# Display the result
print(grouped_data)
```

Then, it was integrated into our original dataset so the cities can be classified into states according to their zip codes.

```
# Create a function to check if a postal code falls within a range
def check_range(postcode, zip_range):
    try:
        start, end = map(int, zip_range.split('-'))
        return start <= int(postcode) <= end
    except ValueError:
        return False

# Apply the function to find the matching range for each 'Postcode'
data['matching_range'] = data['Postcode'].apply(lambda x: next((zip_range for zip_range in grouped_data['zip_range'] if check_range(x, zip_range)), None))

# Merge datasets based on the 'matching_range' column
merged_data = pd.merge(data, grouped_data, left_on='matching_range', right_on='zip_range', how='left')

# Drop unnecessary columns
merged_data = merged_data.drop('matching_range', axis=1)

# Display the result
print(merged_data)
```

After running a check for empty columns, there were some rows which were in the “United States” category that has yet to be classified into a state. After doing manual checking, these rows are all from “El Paso” under the “City” column. A quick research online tells us that this city is located in Texas, so we added Texas to the “state_name” of these columns replacing the original “NaN” value.

```
print("Rows with 'El Paso' before replacement:")
print(merged_data[merged_data['City'] == 'El Paso'])

# After going through the rows with "El Paso", we find that all the "United States" rows with empty values for state_name were missed during the merging of data and grouped
# Replace 'state_name' values
merged_data.loc[merged_data['City'] == 'El Paso', 'state_name'] = 'Texas'

# Print rows with 'El Paso' after the replacement
print("\nRows with 'El Paso' after replacement:")
print(merged_data[merged_data['City'] == 'El Paso'])
```

Then we also noticed there is a typo error in the 738th column, where there was a Chinese character among the value under “City” column. We fixed the typo error and the pre-processing was complete.

```
print(merged_data.loc[738])
```

```
Product ID      L-D-2.5
Coffee Type      Lib
Roast Type       D
Size            2.5
Unit Price      29.785
Price per 100g   1.1914
Profit          3.87205
Customer ID     48873-84433-PN
City            Dún Laoghaire
Country         Ireland
Postcode        A96
Loyalty Card     No
Order ID        RGJ-12544-083
Order Date      2020-02-12
Quantity        3
state_name      NaN
zip_range       NaN
Name: 738, dtype: object
```

```
# city name at index 738 have error, after checking it should be Dún Laoghaire
merged_data.at[738, 'City'] = 'Dún Laoghaire'
print(merged_data.loc[738])
```

```
Product ID      L-D-2.5
Coffee Type      Lib
Roast Type       D
Size            2.5
Unit Price      29.785
Price per 100g   1.1914
Profit          3.87205
Customer ID     48873-84433-PN
City            Dún Laoghaire
Country         Ireland
Postcode        A96
Loyalty Card     No
Order ID        RGJ-12544-083
Order Date      2020-02-12
Quantity        3
state_name      NaN
zip_range       NaN
Name: 738, dtype: object
```

Lastly, the file was saved for preparation of doing data analysis, data analytics and data visualization.

```
merged_data.to_csv('D:\\2023_Sean\\XMUM\\2023\\pre-processed_data.csv', index=False)
```

Our target of the data analysis, data analytics and data visualization are to answer to following objectives:

1) Find the correlation of unit price, price per 100g, profit, and quantity.

1a) Is a higher unit price equivalent to more overall profit? Or will a high unit price affect the quantity bought and lower the overall profit?

1b) What is the unit price range that has a high quantity of sales?

2) Finding anomalies in quantity sales. If there are none, we can compare the sales

2a) Analysing profit mean and total sales

2b) Is the most profitable coffee type also the most sold coffee type?

3) Time analysis of total sales

3a) Total sales and sales trend each year

3b) Prediction of sales (quantity)

3c) What coffee type is trending each year?

4) Analyse country sales on coffee bean

4a) How many orders and sales from each country? If there is a country with high sales/orders, do more detailed analysis/analytics on it

4b) What coffee type/roast type/size does each country prefer?

5) Loyalty card

5a) How many customers have loyalty card and how many don't?

5b) Do customers with loyalty card buy more?

3.0 Data Analysis and Data Analytics with Visualization:

The data set has been preprocessed, and we got a cleaned data set. We will proceed to do some Data Analysis and Data Analytics.

First, we import our data and have a look.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

df_pre_processed_data=pd.read_csv('D:\\2023_Seal\\XMUM\\2023\\pre-processed_data.csv')
print(df_pre_processed_data )
```

```
display(df_pre_processed_data)
```

	Product ID	Coffee Type	Roast Type	Size	Unit Price	Price per 100g	Profit	Customer ID	City	Country	Postcode	Loyalty Card	Order ID	Order Date	Quantity	state_name	zip_range
0	A-L-0.2	Ara	L	0.2	3.885	1.9425	0.34965	36078-91009-WU	San Bernardino	United States	92415	Yes	VMW-26889-781	2019-06-28	2	California	90001-96161
1	A-L-0.2	Ara	L	0.2	3.885	1.9425	0.34965	08847-29858-HN	Stockton	United States	95205	Yes	F/Q-60035-234	2021-04-05	2	California	90001-96161
2	A-L-0.2	Ara	L	0.2	3.885	1.9425	0.34965	47939-53158-LS	Nashville	United States	37215	No	TKL-20738-660	2021-10-02	1	Georgia	30002-39897
3	A-L-0.2	Ara	L	0.2	3.885	1.9425	0.34965	68412-11126-VJ	Washington	United States	20520	Yes	LOO-35324-159	2020-05-19	4	District of Columbia	20001-20591
4	A-L-0.2	Ara	L	0.2	3.885	1.9425	0.34965	43974-44760-QJ	New York City	United States	10105	No	MDC-03318-645	2022-04-30	2	New York	6390-14905
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
995	E-D-2.5	Exc	D	2.5	27.945	1.1178	3.07395	95152-82155-VQ	Austin	United States	78726	No	NJR-03699-189	2019-10-08	1	Texas	73960-79938
996	E-D-2.5	Exc	D	2.5	27.945	1.1178	3.07395	34419-18068-AG	Portland	United States	97271	No	TNR-84447-052	2019-01-09	4	Oregon	97001-97920
997	E-D-2.5	Exc	D	2.5	27.945	1.1178	3.07395	86579-92122-OC	Oklahoma City	United States	73129	Yes	HEL-86709-449	2022-06-10	1	Oklahoma	73001-74966
998	E-D-2.5	Exc	D	2.5	27.945	1.1178	3.07395	76209-39601-ZR	London	United Kingdom	EC3M	Yes	ORX-57454-917	2021-09-06	3	NaN	NaN
999	E-D-2.5	Exc	D	2.5	27.945	1.1178	3.07395	98476-63654-CG	Madison	United States	53726	Yes	MVV-19034-198	2020-05-31	6	Wisconsin	53001-54986

1000 rows × 17 columns

We printed the information of the data set to get better understanding about our data set. Then get summary statistics for numerical columns.

```
#Get summary statistics for numerical columns.
```

```
print(df_pre_processed_data.describe())
```

	Size	Unit Price	Price per 100g	Profit	Quantity
count	1000.000000	1000.000000	1000.000000	1000.000000	1000.000000
mean	1.037300	12.908080	1.438336	1.301375	3.551000
std	0.884025	9.772455	0.365525	1.128052	1.681733
min	0.200000	2.685000	0.823400	0.161100	1.000000
25%	0.500000	5.370000	1.191400	0.505050	2.000000
50%	0.500000	8.950000	1.366200	0.980100	4.000000
75%	1.000000	15.850000	1.650000	1.649100	5.000000
max	2.500000	36.455000	2.377500	4.739150	6.000000

A) Doing some simple analysis and analytics to get an idea

Find the top 5 customers by coffee purchase quantity, the top 5 coffee products by purchase quantity.

And the top 5 orders by purchase quantity.

Top 5 customers by quantity:

	Customer ID	Quantity
157	16880-78077-FB	22
865	94091-86957-HX	18
158	16982-35708-BZ	18
451	51427-89175-QJ	17
595	66458-91190-YC	17

```
#Prints the top 5 customers by coffee purchase quantity.  
# Customer Analysis  
customer_analysis = df_pre_processed_data.groupby('Customer ID')['Quantity'].sum().reset_index()  
customer_analysis.sort_values(by='Quantity', ascending=False, inplace=True)  
print("Top 5 customers by quantity:")  
print(customer_analysis.head())
```

```
#Prints the top 5 coffee products by purchase quantity.
```

```
# Product Analysis  
product_analysis = df_pre_processed_data.groupby('Product ID')['Quantity'].sum().reset_index()  
product_analysis.sort_values(by='Quantity', ascending=False, inplace=True)  
print("Top 5 products by quantity:")  
print(product_analysis.head())
```

Top 5 products by quantity:

	Product ID	Quantity
40	R-L-0.2	100
9	A-M-0.5	99
11	A-M-2.5	97
10	A-M-1	97
0	A-D-0.2	97

Top 5 orders by quantity:

	Order ID	Quantity
497	NOP-21394-646	18
841	WOQ-36015-429	17
563	PKN-19556-918	16
100	CTE-31437-326	15
382	KHO-27106-786	12

```
#Prints the top 5 orders by purchase quantity.  
# Order Analysis  
order_analysis = df_pre_processed_data.groupby('Order ID')['Quantity'].sum().reset_index()  
order_analysis.sort_values(by='Quantity', ascending=False, inplace=True)  
print("Top 5 orders by quantity:")  
print(order_analysis.head())
```

Customer Analysis:

Identifies the top customers based on the total quantity of products they've purchased. Useful for recognizing high-value customers.

Product Analysis:

Highlights the top-selling products based on the total quantity sold. Helps identify popular products.

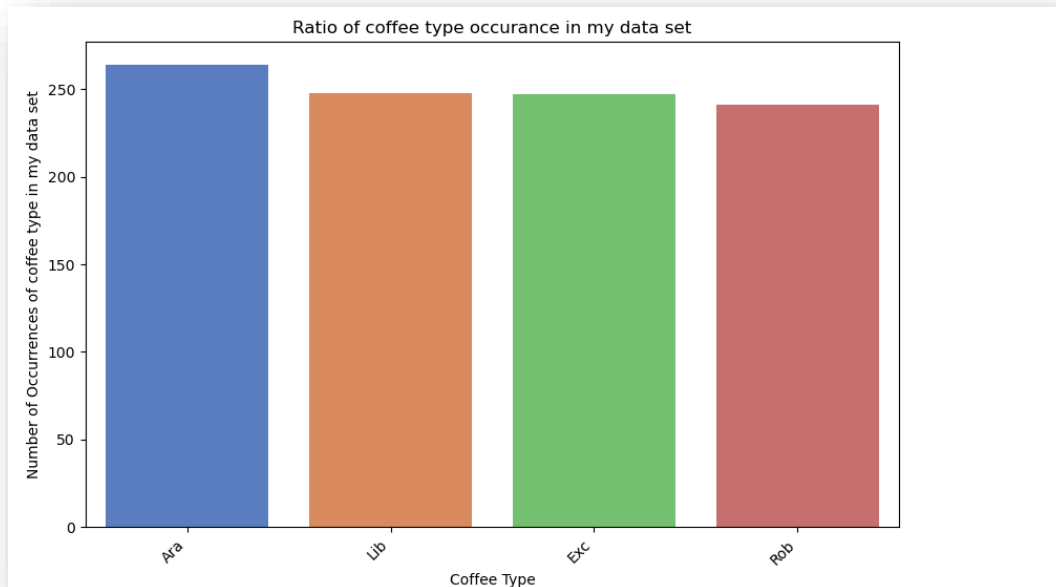
Order Analysis:

Identifies the top orders based on the total quantity of products ordered. Useful for understanding the impact of specific orders on overall quantity.

Ratio of coffee type occurrence in our data set.

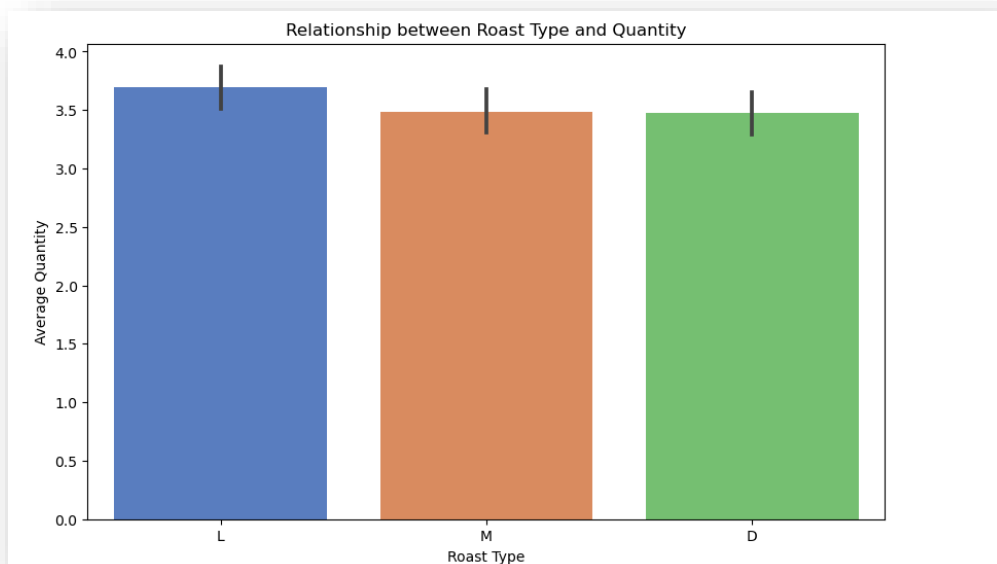
```
# Count the occurrences of each coffee type
coffee_type_counts = df_pre_processed_data['Coffee Type'].value_counts()

# Plot a bar plot for the most popular coffee types
plt.figure(figsize=(10, 6))
sns.barplot(x=coffee_type_counts.index, y=coffee_type_counts.values, palette='muted')
plt.title('Ratio of coffee type occurrence in my data set')
plt.xlabel('Coffee Type')
plt.ylabel('Number of Occurrences of coffee type in my data set')
plt.xticks(rotation=45, ha='right')
plt.show()
```



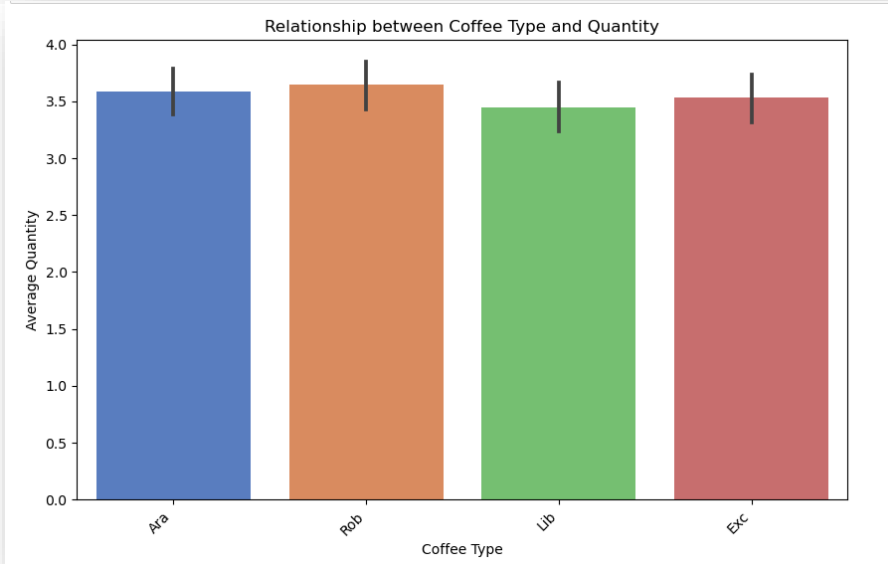
The result indicates that the coffee types appear at an average rate in my data set, so this data set is reliable. The coffee types are Arabica, Liberica, Excelsa and Robusta.

```
# Create a bar plot for the relationship between Roast Type and Quantity
plt.figure(figsize=(10, 6))
sns.barplot(x='Roast Type', y='Quantity', data=df_pre_processed_data, palette='muted')
plt.title('Relationship between Roast Type and Quantity')
plt.xlabel('Roast Type')
plt.ylabel('Average Quantity')
plt.show()
```



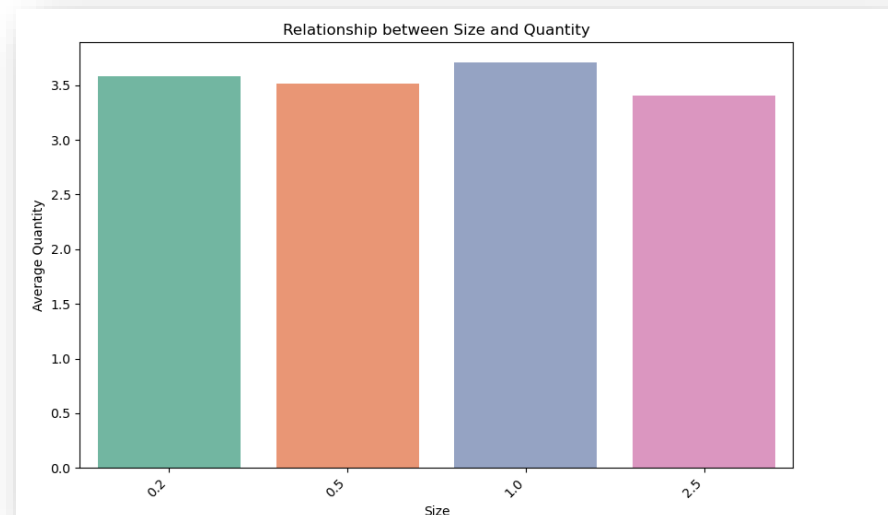
L stands for light, M stands for medium, D stands for Dark. Based on the result, L type is the most popular type of roast. On average, people will buy more light roasted coffee beans at once.

```
# Create a bar plot for the relationship between Coffee Type and Quantity
plt.figure(figsize=(10, 6))
sns.barplot(x='Coffee Type', y='Quantity', data=df_pre_processed_data, palette='muted')
plt.title('Relationship between Coffee Type and Quantity')
plt.xlabel('Coffee Type')
plt.ylabel('Average Quantity')
plt.xticks(rotation=45, ha='right')
plt.show()
```



Based on the result, Rob coffee bean has the most loyal customers by a small margin. This is not significant, and will not be counted as a useful insight. To understand if customers prefer a single type of coffee bean, we have to do further analysis.

```
# Create a bar plot for the relationship between Size and Quantity
plt.figure(figsize=(10, 6))
sns.barplot(x='Size', y='Quantity', data=df_pre_processed_data, palette='Set2', errorbar=None)
plt.title('Relationship between Size and Quantity')
plt.xlabel('Size')
plt.ylabel('Average Quantity')
plt.xticks(rotation=45, ha='right')
plt.show()
```



Based on the result, 1.0 is the most popular size of coffee package. We can make an assumption that this could be the standard size of most coffee bean products.

B) Main objectives to find out

1) Find the correlation of unit price, price per 100g, profit, and quantity.

1a) Is a higher unit price equivalent to more overall profit? Or will a high unit price affect the quantity bought and lower the overall profit?

```
# Selecting the relevant columns
selected_columns = ['Unit Price', 'Price per 100g', 'Profit']
df_selected = df_pre_processed_data[selected_columns]

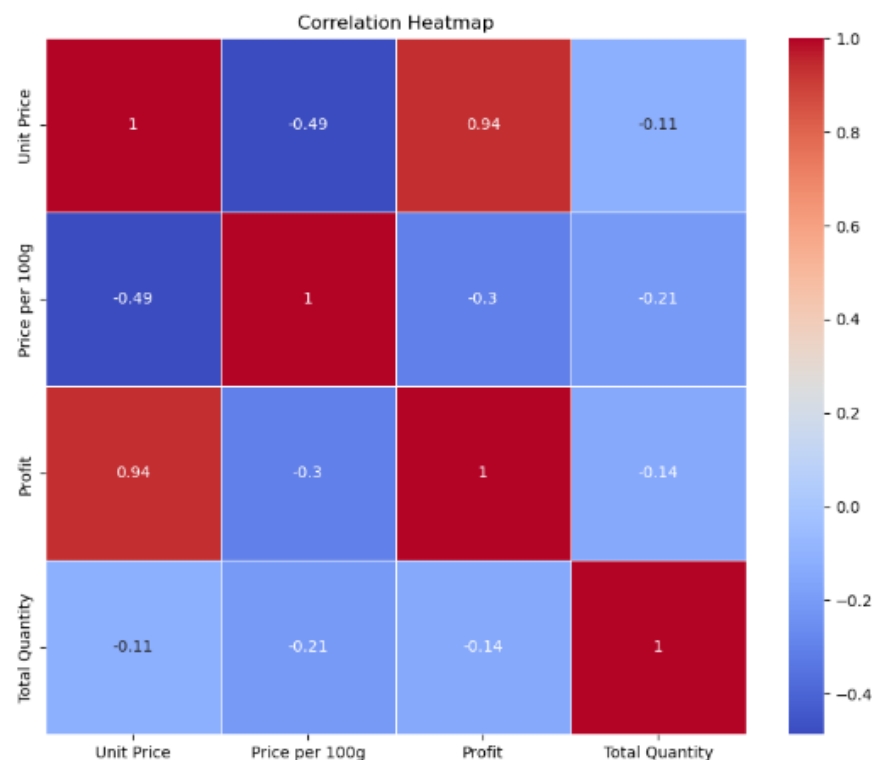
# Merge df_selected with agg_df on a common column (e.g., 'Unit Price')
merged_data = pd.merge(df_selected, agg_df, on='Unit Price', how='inner')

# Calculate the correlation matrix
correlation_matrix = merged_data.corr()

# Create a heatmap
plt.figure(figsize=(10, 8))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', linewidths=.5)
plt.title('Correlation Heatmap')
plt.show()
```

Correlation Matrix:

	Unit Price	Price per 100g	Profit	Quantity
Unit Price	1.000000	-0.486816	0.940073	-0.042778
Price per 100g	-0.486816	1.000000	-0.301366	0.008045
Profit	0.940073	-0.301366	1.000000	-0.053275
Quantity	-0.042778	0.008045	-0.053275	1.000000



From the correlation matrix, we can see that profit and unit price have strong positive correlations which mean that the higher the unit price, the higher the profit.

Since we found out that quantity and unit price have weak negative correlation, there is little correlation between the two.

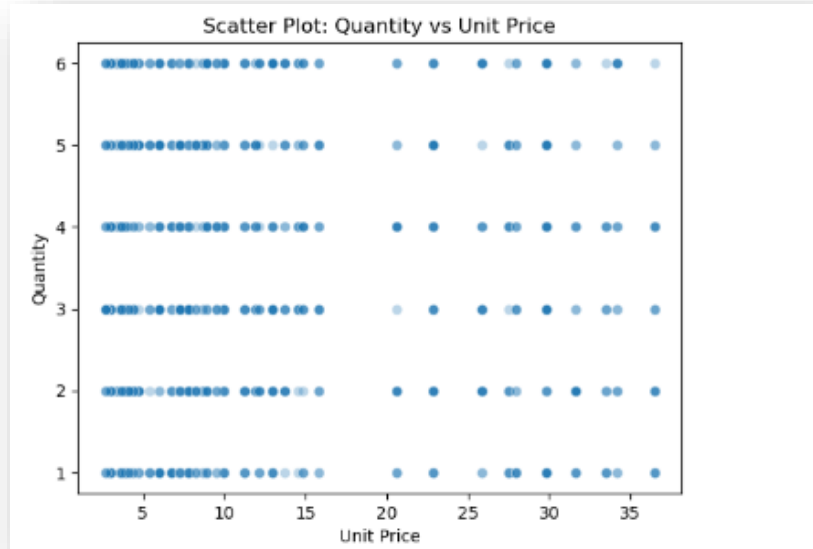
From these, we can conclude that a high unit price doesn't necessarily turn customers away from buying the product. Therefore, it is possible for us to raise the unit price moderately to earn a higher profit.

1b) What is the unit price range that has a high quantity of sales?

```
# Generate the scatter plot
sns.scatterplot(x='Unit Price', y='Quantity', data=df_pre_processed_data, alpha=0.3)

# Add title and labels
plt.title('Scatter Plot: Quantity vs Unit Price')
plt.xlabel('Unit Price')
plt.ylabel('Quantity')

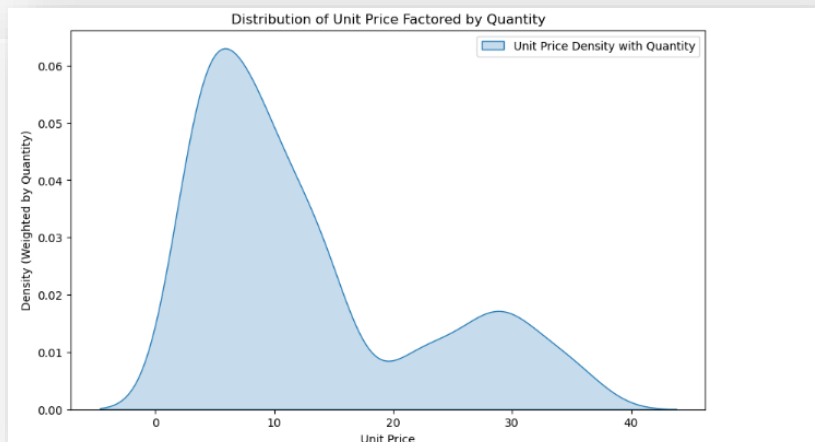
# Display the plot
plt.show()
```



Through the scatter plot, we can see that the highest distribution of sales quantity is at the unit price range between 0-15, the unit price range between 20-40 is moderate, and there is little to no distribution of quantity of sales at the unit price range between 15-20. For a better understanding, please refer to the following distribution graph.

```
plt.figure(figsize=(10, 6))

# Use the 'Quantity' as weights for the kernel density estimation of 'Unit Price'
sns.kdeplot(data=df_pre_processed_data, x='Unit Price', weights='Quantity', shade=True, label='Unit Price Density with Quantity')
plt.title('Distribution of Unit Price Factored by Quantity')
plt.xlabel('Unit Price')
plt.ylabel('Density (Weighted by Quantity)')
plt.legend()
plt.show()
```



From this, we can say that customers will always choose the cheapest option first to save money. If they have extra money, they are likely to choose more expensive options. Very little people will choose to buy a product which unit price falls near the median. This is very interesting as we initially expected

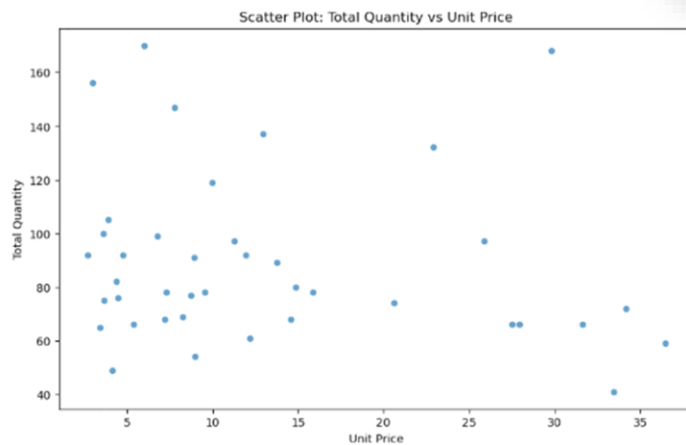
the sales quantity to reach the maximum at the range between 15-25 like a normal distribution does. This phenomenon could probably be explained by the Prospect Theory where people prefer to buy a more expensive coffee type rather than suffering the loss of getting a subpar coffee type. The seller should put more focus on giving expensive products more exposure so more people will buy them.

```
# Create a scatter plot
plt.figure(figsize=(10, 6))
sns.scatterplot(x='Unit Price', y='Total Quantity', data=agg_df, alpha=0.7)
plt.title('Scatter Plot: Total Quantity vs Unit Price')
plt.xlabel('Unit Price')
plt.ylabel('Total Quantity')
plt.show()
```

```
# Aggregate data to get total quantity for each unique unit price
agg_df = df_pre_processed_data.groupby('Unit Price')['Quantity'].sum().reset_index()

# Rename a single column
agg_df.rename(columns={'Quantity': 'Total Quantity'}, inplace=True)

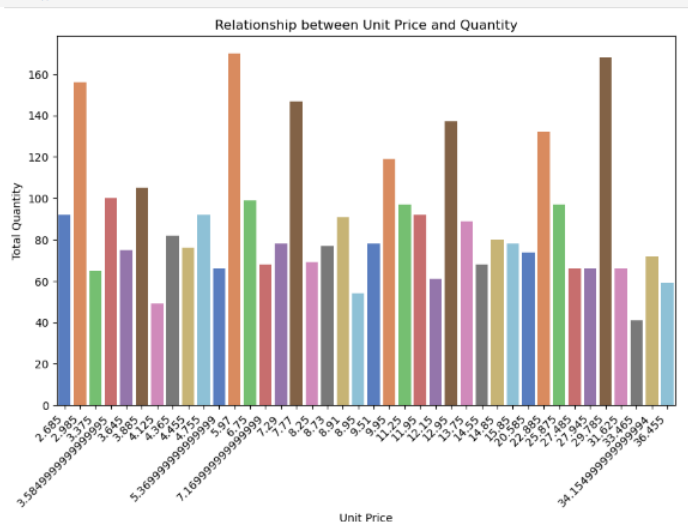
display(agg_df)
```



```
# Create a bar plot for the relationship between unit price and Quantity
plt.figure(figsize=(10, 6))
sns.barplot(x='Unit Price', y='Total Quantity', data=agg_df, palette='muted', errorbar=None)

# Format x-axis ticks to two decimal places with rounding
def format_func(value, tick_number):
    rounded_value = round(value, 2)
    return f'{rounded_value:.2f}'

plt.title('Relationship between Unit Price and Quantity')
plt.xlabel('Unit Price')
plt.ylabel('Total Quantity')
plt.xticks(rotation=45, ha='right')
plt.show()
```



	Unit Price	Total Quantity
0	2.685	92
1	2.985	156
2	3.375	65
3	3.585	100
4	3.645	75
5	3.885	105
6	4.125	49
7	4.365	82
8	4.455	76
9	4.755	92
10	5.370	66
11	5.970	170
12	6.750	99
13	7.170	68
14	7.290	78
15	7.770	147
16	8.250	69
17	8.730	77
18	8.910	91
19	8.950	54
20	9.510	78
21	9.950	119
22	11.250	97
23	11.950	92
24	12.150	61
25	12.950	137
26	13.750	89
27	14.550	68
28	14.850	80
29	15.850	78
30	20.585	74
31	22.885	132
32	25.875	97
33	27.485	66
34	27.945	66
35	29.785	168
36	31.625	66
37	33.465	41
38	34.155	72
39	36.455	59

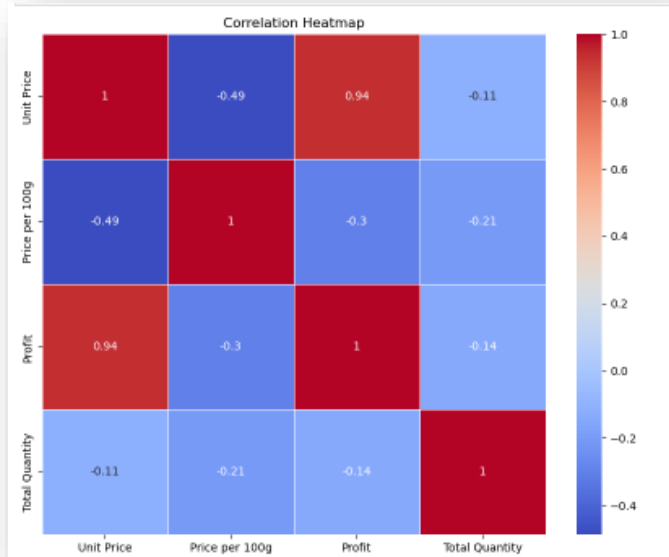
```
# Selecting the relevant columns
selected_columns = ['Unit Price', 'Price per 100g', 'Profit']
df_selected = df_pre_processed_data[selected_columns]

# Merge df_selected with agg_df on a common column (e.g., 'Unit Price')
merged_data = pd.merge(df_selected, agg_df, on='Unit Price', how='inner')

# Calculate the correlation matrix
correlation_matrix = merged_data.corr()

# Create a heatmap
plt.figure(figsize=(10, 8))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', linewidths=.5)
plt.title('Correlation Heatmap')
plt.show()
```

```
correlation = agg_df['Unit Price'].corr(agg_df['Total Quantity'])
print(f"Correlation between Total Quantity and Unit Price: {correlation}")
Correlation between Total Quantity and Unit Price: -0.15433218425262388
```



The above were done to see if we could find a relationship between total quantity sold and unit price. However, we were not able to achieve this and we only managed to find the unit prices with the highest quantity sold. Therefore, we did not proceed with any more analysis for this problem.

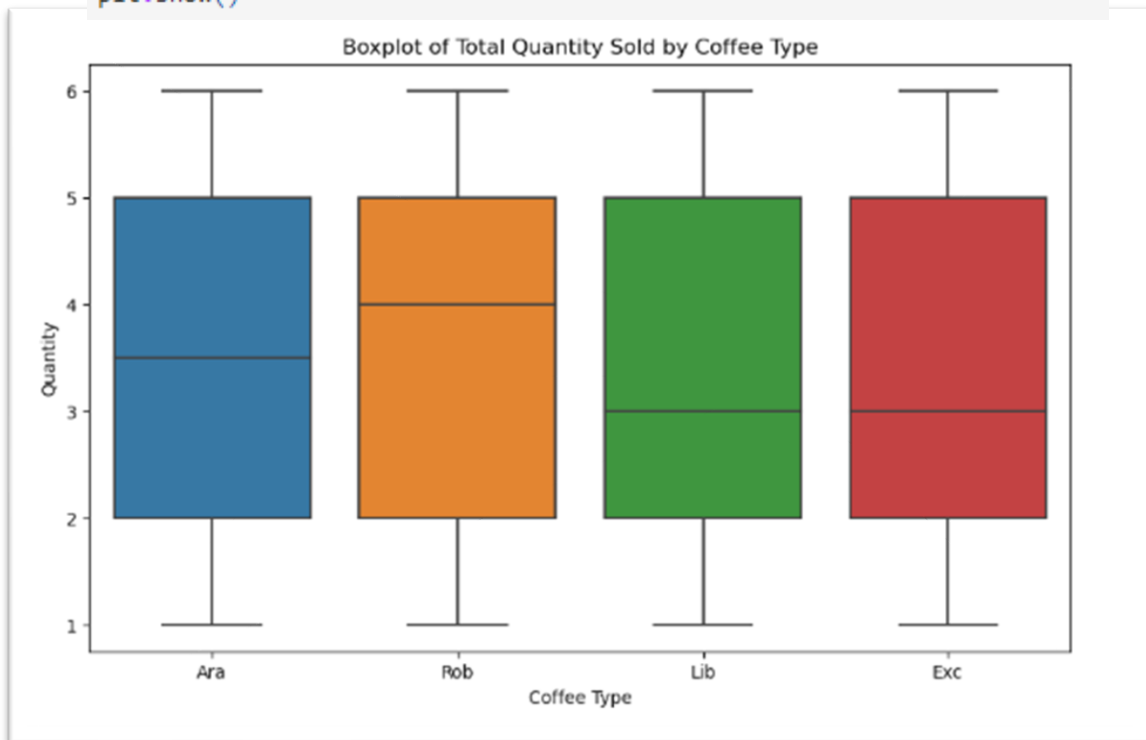
2) Finding anomalies in quantity sales. If there are none, we can compare the sales

```
from scipy.stats import f_oneway

# Descriptive statistics
summary_stats = df_pre_processed_data.groupby('Coffee Type')['Quantity'].describe()
print(summary_stats)
```

Coffee Type	count	mean	std	min	25%	50%	75%	max
Ara	264.0	3.587121	1.722140	1.0	2.0	3.5	5.0	6.0
Exc	247.0	3.530364	1.695909	1.0	2.0	3.0	5.0	6.0
Lib	248.0	3.443548	1.653373	1.0	2.0	3.0	5.0	6.0
Rob	241.0	3.643154	1.654931	1.0	2.0	4.0	5.0	6.0

```
# Visualization: Boxplot
plt.figure(figsize=(10, 6))
sns.boxplot(x='Coffee Type', y='Quantity', data=df_pre_processed_data)
plt.title('Boxplot of Total Quantity Sold by Coffee Type')
plt.show()
```



```
# Statistical Test: ANOVA
# Checking if there are significant differences in mean quantities sold across coffee types
coffee_types = df_pre_processed_data['Coffee Type'].unique()
data_per_coffee = [df_pre_processed_data[df_pre_processed_data['Coffee Type'] == coffee]['Quantity'] for coffee in coffee_types]

f_statistic, p_value = f_oneway(*data_per_coffee)

print(f"ANOVA F-statistic: {f_statistic}")
print(f"ANOVA p-value: {p_value}")

# If p-value < 0.05, we reject the null hypothesis
if p_value < 0.05:
    print("There is a significant difference in mean quantities sold across coffee types.")
else:
    print("There is no significant difference in mean quantities sold across coffee types.")
```

```
ANOVA F-statistic: 0.6309842632354191
ANOVA p-value: 0.5950997661866411
There is no significant difference in mean quantities sold across coffee types.
```

We concluded that there is no significant difference in mean quantities, which means the data is overall fair and not biased when we try to analyse the sales and profits. We can proceed to analyse the sales and profit of the coffee types.

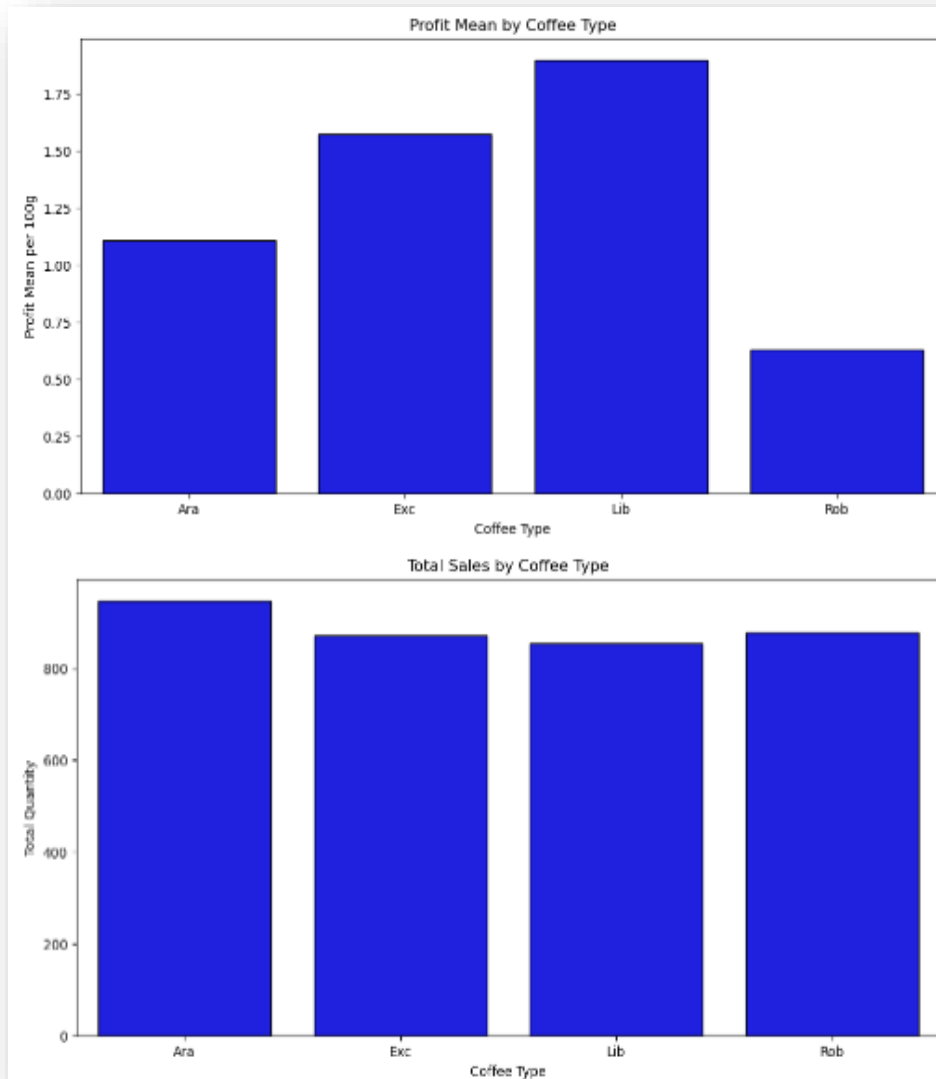
2a) Analysing profit mean and total sales

```
# 2a) Profit and Sales by Coffee Type
def plot_bar(data, x, y, title, ylabel):
    plt.figure(figsize=(12, 6))
    sns.barplot(x=x, y=y, data=data, color="blue", edgecolor="black", errorbar=None)
    plt.title(title)
    plt.xlabel(x)
    plt.ylabel(ylabel)
    plt.show()

# Calculate the "Profit Mean" and "Total Quantity" for each Coffee Type
profit_mean_data = df_pre_processed_data.groupby('Coffee Type')['Profit'].mean().reset_index()
total_quantity_data = df_pre_processed_data.groupby('Coffee Type')['Quantity'].sum().reset_index()

# Rename the columns for clarity
profit_mean_data.rename(columns={'Profit': 'Profit Mean'}, inplace=True)
total_quantity_data.rename(columns={'Quantity': 'Total Quantity'}, inplace=True)

# Plotting
plot_bar(profit_mean_data, 'Coffee Type', 'Profit Mean', 'Profit Mean by Coffee Type', 'Profit Mean per 100g')
plot_bar(total_quantity_data, 'Coffee Type', 'Total Quantity', 'Total Sales by Coffee Type', 'Total Quantity')
```



Liberica has the highest profit mean, which means by selling the same quantity of different coffee bean products, Liberica brings in the most profit. As for the second graph, it shows that Ara has a slight advantage in total quantity sold compared to other coffee types. After plotting the two graphs separately, we will do a plot which combines total sales and total profit.

```
# 2b) Coffee Type with highest profit and highest sales
Ct = df_pre_processed_data["Coffee Type"].unique()
x1 = sorted(Ct)

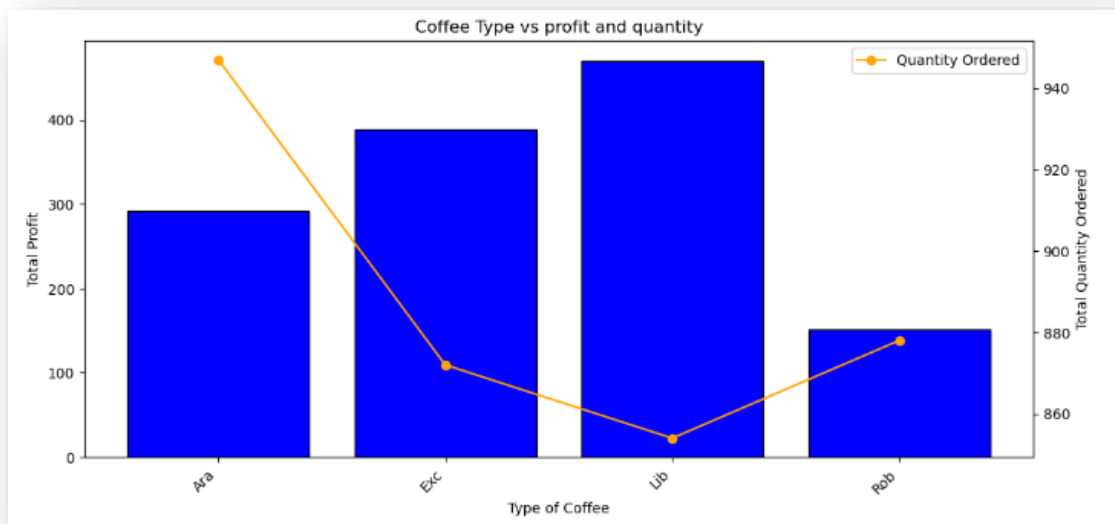
y2 = df_pre_processed_data.groupby(["Coffee Type"])["Profit"].sum()
y3 = df_pre_processed_data.groupby(["Coffee Type"])["Quantity"].sum()

fig, ax1 = plt.subplots(figsize=(12, 6))
ax2 = ax1.twinx()

ax1.bar(x1, y2, color="blue", edgecolor="black", label='Total Profit')
ax2.plot(x1, y3, "o-", color="orange", label='Quantity Ordered')

ax1.set_xlabel('Type of Coffee')
ax1.set_ylabel('Total Profit')
ax2.set_ylabel("Total Quantity Ordered")

plt.title("Coffee Type vs profit and quantity")
fig.autofmt_xdate(rotation=45)
plt.legend(loc='best')
plt.show()
```



From this graph, we have an interesting discovery. Liberica coffee beans has the lowest demand (lowest quantity ordered), yet the total profit it brings is the lowest. On the other hand, Arabica coffee beans has the highest demand (highest quantity ordered), but its total profit is ranked second lowest. We can therefore conclude that the seller should put more focus into promoting Liberica coffee beans as it brings the most profit.

3) Time analysis of total sales

3a) Total sales and sales trend each year

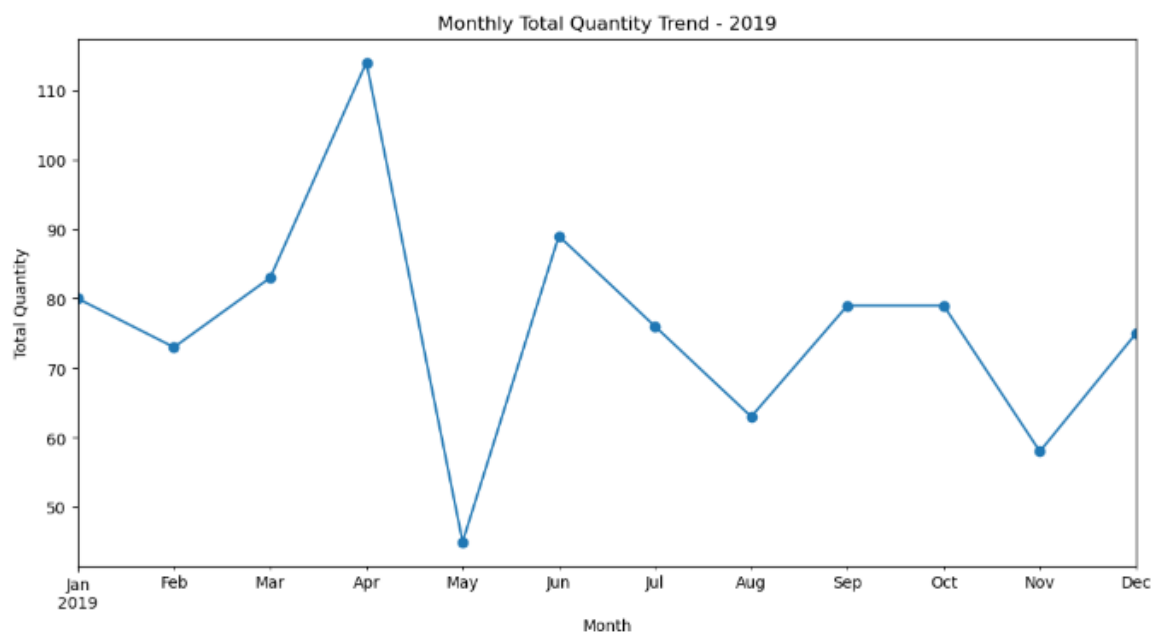
```
# 3a) Total sales each year
df_pre_processed_data['Order Date'] = pd.to_datetime(df_pre_processed_data['Order Date'])

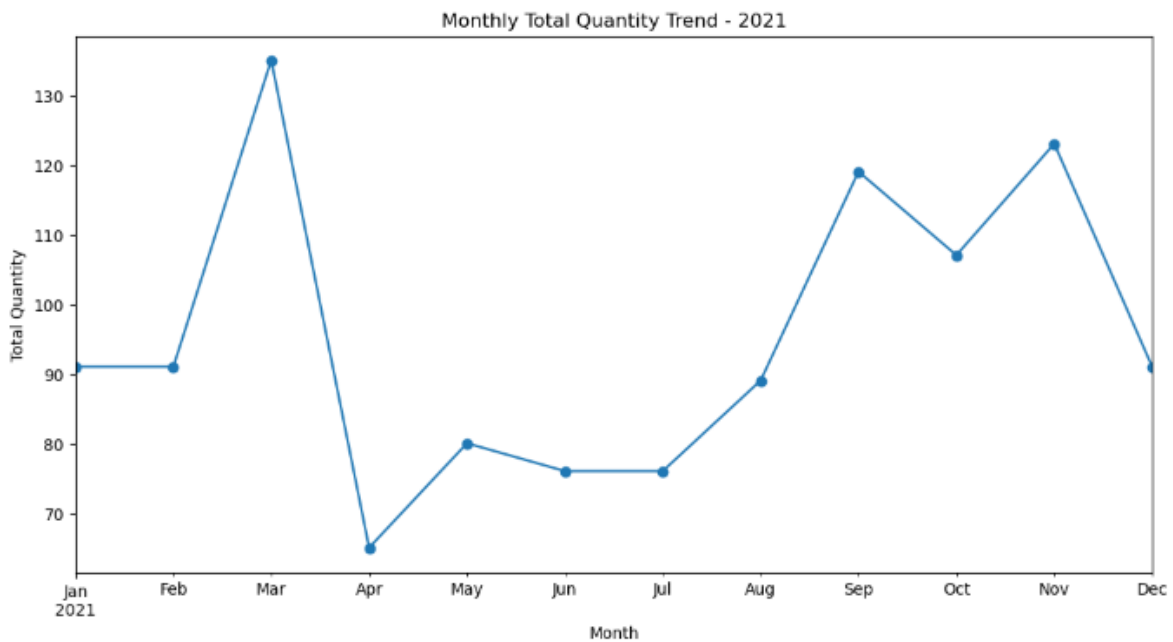
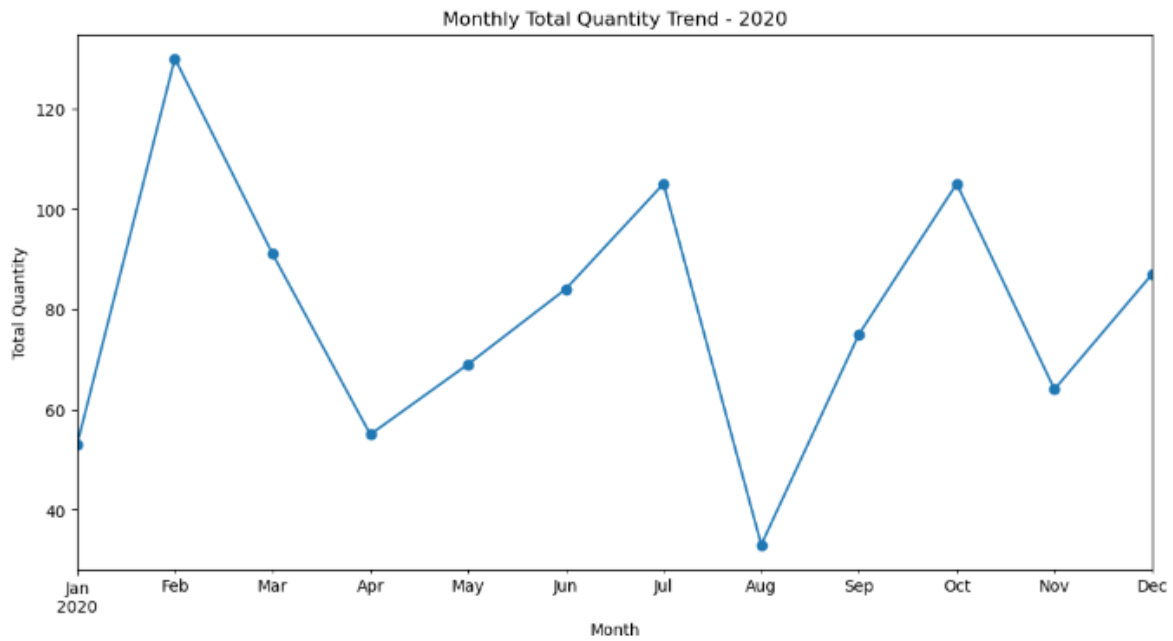
# Filter data for each year
df_2019 = df_pre_processed_data[df_pre_processed_data['Order Date'].dt.year == 2019]
df_2020 = df_pre_processed_data[df_pre_processed_data['Order Date'].dt.year == 2020]
df_2021 = df_pre_processed_data[df_pre_processed_data['Order Date'].dt.year == 2021]

# Monthly total quantity trend for 2019
monthly_trend_2019 = df_2019.resample('M', on='Order Date')['Quantity'].sum()
plt.figure(figsize=(12, 6))
monthly_trend_2019.plot(marker='o')
plt.title('Monthly Total Quantity Trend - 2019')
plt.xlabel('Month')
plt.ylabel('Total Quantity')
plt.show()

# Monthly total quantity trend for 2020
monthly_trend_2020 = df_2020.resample('M', on='Order Date')['Quantity'].sum()
plt.figure(figsize=(12, 6))
monthly_trend_2020.plot(marker='o')
plt.title('Monthly Total Quantity Trend - 2020')
plt.xlabel('Month')
plt.ylabel('Total Quantity')
plt.show()

# Monthly total quantity trend for 2021
monthly_trend_2021 = df_2021.resample('M', on='Order Date')['Quantity'].sum()
plt.figure(figsize=(12, 6))
monthly_trend_2021.plot(marker='o')
plt.title('Monthly Total Quantity Trend - 2021')
plt.xlabel('Month')
plt.ylabel('Total Quantity')
plt.show()
```





We separated the sales quantity by year and plotted them into different graphs to see if we can find a trend. We noticed that the trend is always increasing at the start of the year, then it plummets around April to May, which is a recurring pattern for all three years. This could be due to the expiry date (or “Best if Used before” date) of the coffee bean product being counted in years. Meaning, if you buy a coffee bean product on January 2018, it may expire on the same month two years later, which is January 2020. This could explain the certain spike in sales every year at the same month. Most people probably bought their products at the start of the year initially, so naturally the product will be finished up or expired by the start of a few years later too. Therefore, customers will buy the products at the same month of the year again.

```

df_pre_processed_data['Order Date'] = pd.to_datetime(df_pre_processed_data['Order Date'])

# Calculate monthly trends for each year
monthly_trends = {}
for year in [2019, 2020, 2021]:
    df_year = df_pre_processed_data[df_pre_processed_data['Order Date'].dt.year == year]
    monthly_trend = df_year.resample('M', on='Order Date')['Quantity'].sum()
    monthly_trend.index = monthly_trend.index.month # Set month-based index
    monthly_trends[year] = monthly_trend

# Create the plot
plt.figure(figsize=(12, 6))

# Plot lines in reverse order for correct overlapping
for year in reversed(monthly_trends.keys()):
    plt.plot(monthly_trends[year].index, monthly_trends[year].values, marker='o', label=str(year))

# Add titles and labels
plt.title('Monthly Total Quantity Trend by Year (Overlapping)')
plt.xlabel('Month')
plt.ylabel('Total Quantity')

# Add Legend
plt.legend()

plt.show()

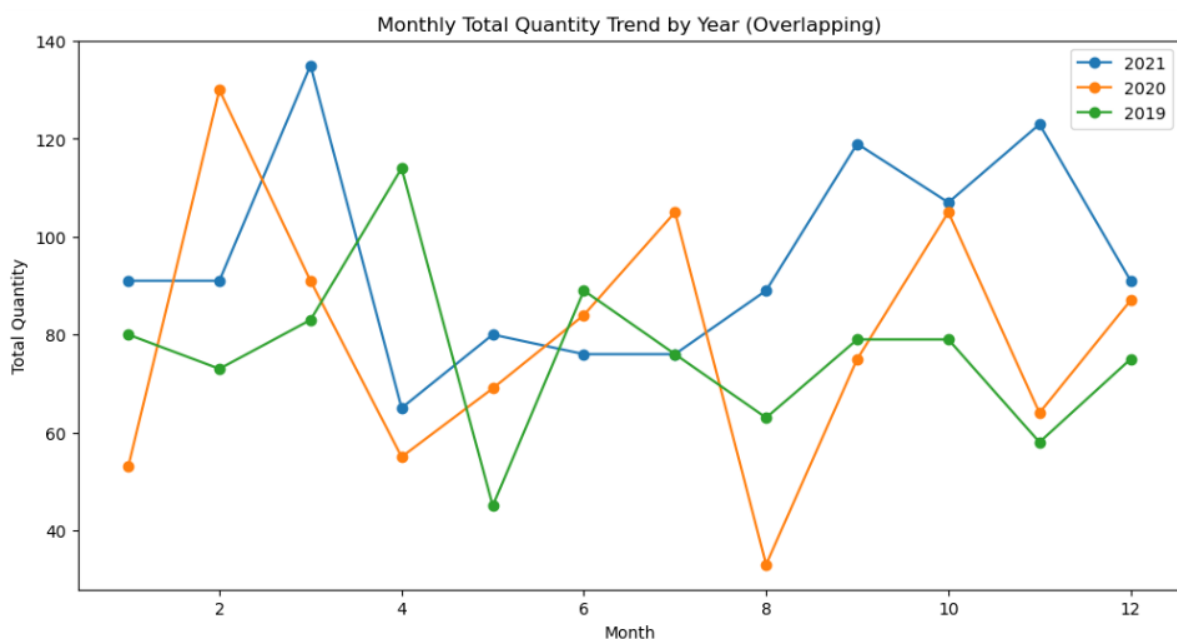
# 3b) Which year has the highest sales? Which year has the lowest sales? Predict 2022 sales if possible.
# Convert 'Order Date' to datetime
df_pre_processed_data['Order Date'] = pd.to_datetime(df_pre_processed_data['Order Date'])

# Filter data for each year
df_2019 = df_pre_processed_data[df_pre_processed_data['Order Date'].dt.year == 2019]
df_2020 = df_pre_processed_data[df_pre_processed_data['Order Date'].dt.year == 2020]
df_2021 = df_pre_processed_data[df_pre_processed_data['Order Date'].dt.year == 2021]

# Calculate total sales for each year
total_sales_2019 = df_2019['Quantity'].sum()
total_sales_2020 = df_2020['Quantity'].sum()
total_sales_2021 = df_2021['Quantity'].sum()

# Print total sales for each year
print(f'Total Sales in 2019: {total_sales_2019}')
print(f'Total Sales in 2020: {total_sales_2020}')
print(f'Total Sales in 2021: {total_sales_2021}')

```



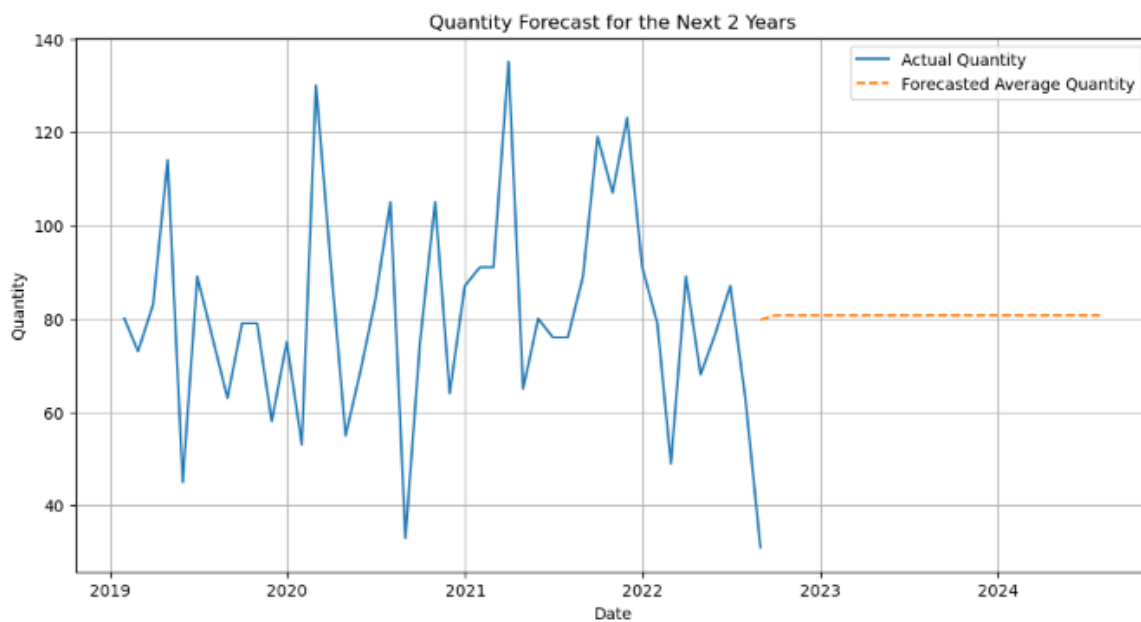
This is overlapping plot of the monthly total quantity trend by year. It is easier to see where the spikes and plummets overlap by doing so.

Total Sales in 2019: 914
Total Sales in 2020: 951
Total Sales in 2021: 1143

We noticed that 2021 had the most sales. This could be due to the economy recovering from the COVID-19 pandemic, resulting in an increase in sales.

3b) Prediction of sales (quantity)

```
from statsmodels.tsa.arima.model import ARIMA
# Here's a simple ARIMA(1,1,1) model. You can adjust the parameters based on your data.
monthly_quantity = df_pre_processed_data.resample('M', on='Order Date')['Quantity'].sum()
model = ARIMA(monthly_quantity, order=(1,1,1))
model_fit = model.fit()
forecast = model_fit.forecast(steps=24) # Forecasting for 2 years (24 months)
future_dates = pd.date_range(start=df_pre_processed_data['Order Date'].max(), periods=24, freq='M')
plt.figure(figsize=(12, 6))
plt.plot(monthly_quantity.index, monthly_quantity.values, label='Actual Quantity')
plt.plot(future_dates, forecast, label='Forecasted Average Quantity', linestyle='--')
plt.title('Quantity Forecast for the Next 2 Years')
plt.xlabel('Date')
plt.ylabel('Quantity')
plt.legend()
plt.grid(True)
plt.show()
```



We tried to perform a prediction on the average quantity of sales for the following years. It is not possible to make a prediction on accurate sales as we do not have all the factors that affect the quantity sold.

3c) What coffee type is trending each year?

```
# 3c) What coffee type is trending each year?
# Assuming df_pre_processed_data is your DataFrame
df_pre_processed_data['Order Date'] = pd.to_datetime(df_pre_processed_data['Order Date'])

# Extract relevant features
df_pre_processed_data['Year'] = df_pre_processed_data['Order Date'].dt.year

# Group data by Year and Coffee Type, summing up the quantities
coffee_trends = df_pre_processed_data.groupby(['Year', 'Coffee Type'])['Quantity'].sum().reset_index()

# Find the coffee type with the highest quantity for each year
trending_coffee_types = coffee_trends.loc[coffee_trends.groupby('Year')['Quantity'].idxmax()]
# Plot the trending coffee types using a bar graph
plt.figure(figsize=(12, 6))

# Color palette for the bars
colors = plt.cm.viridis(np.linspace(0, 1, len(trending_coffee_types['Coffee Type'].unique()))))

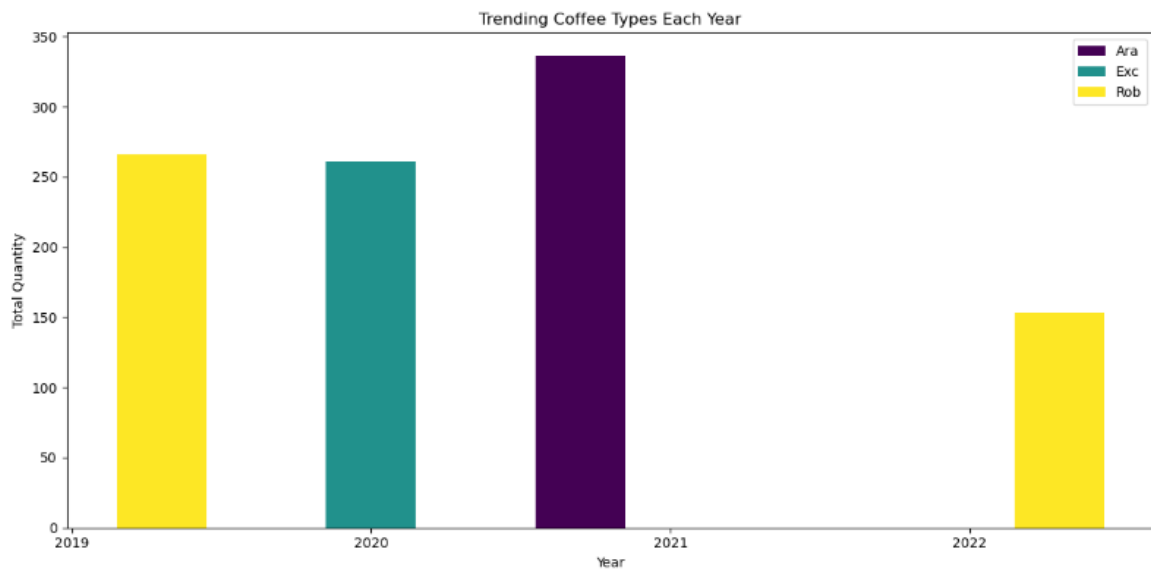
# Bar width for each year
bar_width = 0.3 # Adjusted to be narrower

for idx, (coffee_type, data) in enumerate(trending_coffee_types.groupby('Coffee Type')):
    plt.bar(data['Year'] + idx * bar_width, data['Quantity'], width=bar_width, label=coffee_type, color=colors[idx])

plt.title('Trending Coffee Types Each Year')
plt.xlabel('Year')
plt.ylabel('Total Quantity')

# Adjusting x-axis ticks for better spacing
plt.xticks(trending_coffee_types['Year'].unique() + bar_width * (len(trending_coffee_types['Coffee Type'].unique()) - 1) / 2, labels=trending_coffee_types['Year'].unique())

plt.legend()
plt.tight_layout()
plt.show()
```



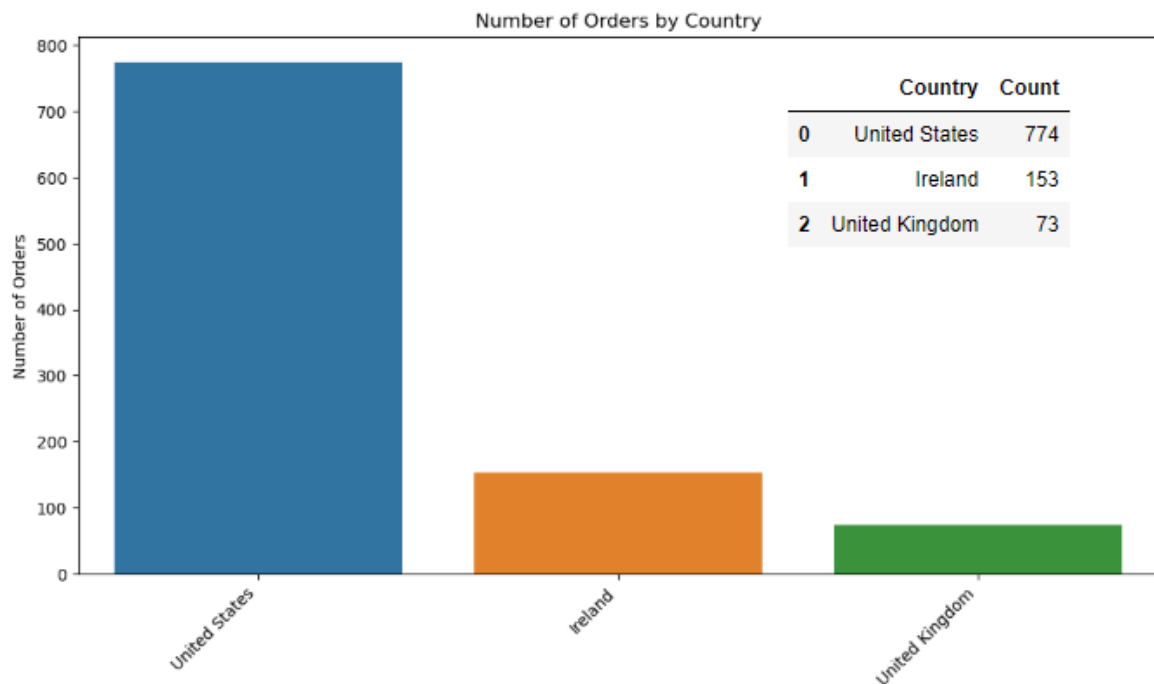
We can see that Liberica is not a popular coffee bean choice. On the other hand, Robusta coffee bean seems to be the overall popular coffee bean choice because it appeared twice in the trending coffee types by year.

4) Analyse country sales on coffee bean

4a) How many orders and sales from each country?

```
# 4a) How many sales from each country?
# Number of orders per country
orders_by_country = df_pre_processed_data['Country'].value_counts()
plt.figure(figsize=(12, 6))
sns.barplot(x=orders_by_country.index, y=orders_by_country.values)
plt.title('Number of Orders by Country')
plt.xlabel('Country')
plt.ylabel('Number of Orders')
plt.xticks(rotation=45, ha='right')
plt.show()
```

```
country_counts = df_pre_processed_data['Country'].value_counts().reset_index()
country_counts.columns = ['Country', 'Count']
display(country_counts)
```



United States contributes most of the orders. However, this could be due to the reason that the seller is a franchised-based business based in United States and had only few branches in Ireland and United Kingdom.

```

# Extract relevant columns
country_sales = df_pre_processed_data[['Country', 'Quantity']]

# Group data by Country and sum up the quantities
total_quantity_per_country = country_sales.groupby('Country')['Quantity'].sum().reset_index()

# Merge with df_country_totals to get the total quantity information
merged_data = pd.merge(total_quantity_per_country, agg_df_3, on='Country', how='left')

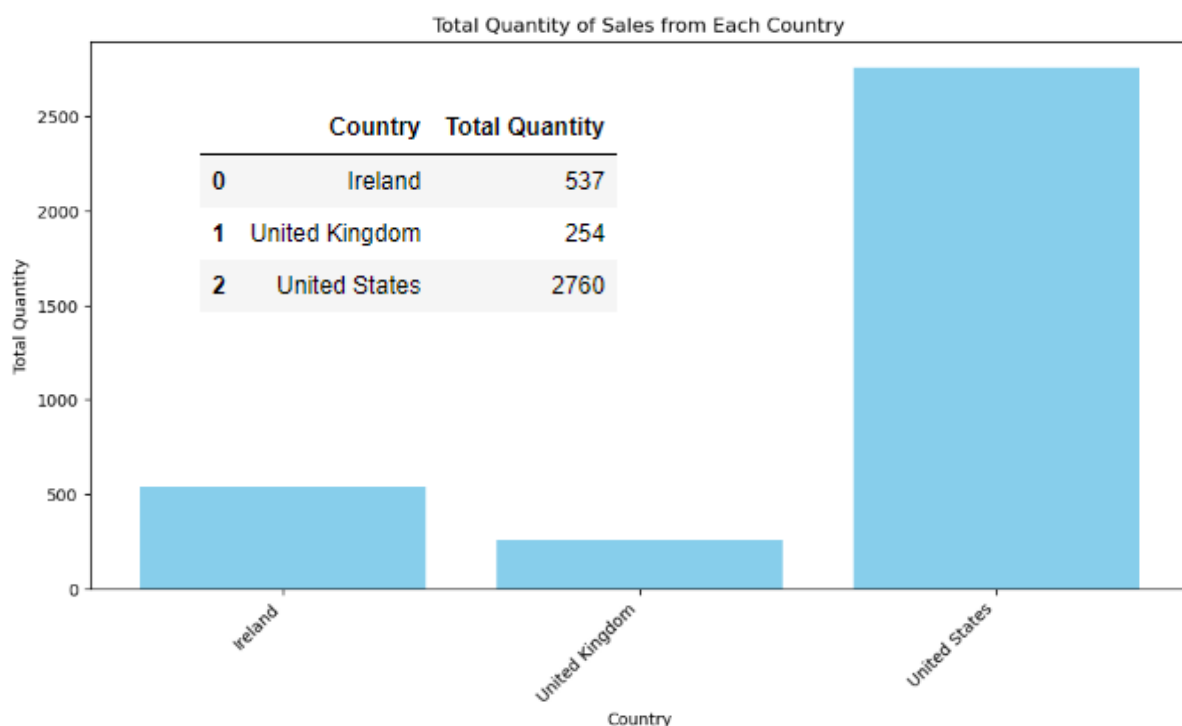
# Plot the total quantity for each country
plt.figure(figsize=(12, 6))
plt.bar(merged_data['Country'], merged_data['Total Quantity'], color='skyblue')
plt.title('Total Quantity of Sales from Each Country')
plt.xlabel('Country')
plt.ylabel('Total Quantity')
plt.xticks(rotation=45, ha='right')
plt.show()

# Aggregate data to get total quantity for each country
agg_df_3 = df_pre_processed_data.groupby('Country')['Quantity'].sum().reset_index()

# Rename a single column
agg_df_3.rename(columns={'Quantity': 'Total Quantity'}, inplace=True)

display(agg_df_3)

```



There is not much useful insight we can get from the graph since the difference in sales is too big. All that we can conclude is most of the coffee bean products is sold in the United States. We have decided to do a further analysis into United States by separating it into states. This is what the integration of the `us_zips.csv` file was for.

```

us_quantity = df_pre_processed_data[df_pre_processed_data['Country'] == 'United States']
max_state_name = None # Initialize to None
max_state_quantity = None # Initialize to None
max_customers_state = None # Initialize to None
max_customers_count = None # Initialize to None

if not us_quantity.empty:
    # Find state with the most quantity
    state_quantity = us_quantity.groupby('state_name')['Quantity'].sum()
    if not state_quantity.empty:
        max_state_name = state_quantity.idxmax()
        max_state_quantity = state_quantity[max_state_name]

    # Find state with the most customers
    customers_count = us_quantity['state_name'].value_counts()
    if not customers_count.empty:
        max_customers_state = customers_count.idxmax()
        max_customers_count = customers_count[max_customers_state]

    # Print the results
    if max_state_name:
        print(f"The state with the most quantity in the US is: {max_state_name} with a quantity of {max_state_quantity}")
    else:
        print("There are no quantity records for states in the US.")

    if max_customers_state:
        print(f"The state with the most customers in the US is: {max_customers_state} with {max_customers_count} customers.")

    # Relationship between Most Customers and Most Sales
    if max_state_name and max_customers_state:
        print(f"Do the state with the most customers and the state with the most quantity match? {'Yes' if max_state_name == max_customers_state else 'No'}")
else:
    print("There are no US records in the dataset.")

```

The state with the most quantity in the US is: Texas with a quantity of 316
The state with the most customers in the US is: California with 90 customers.
Do the state with the most customers and the state with the most quantity match? No

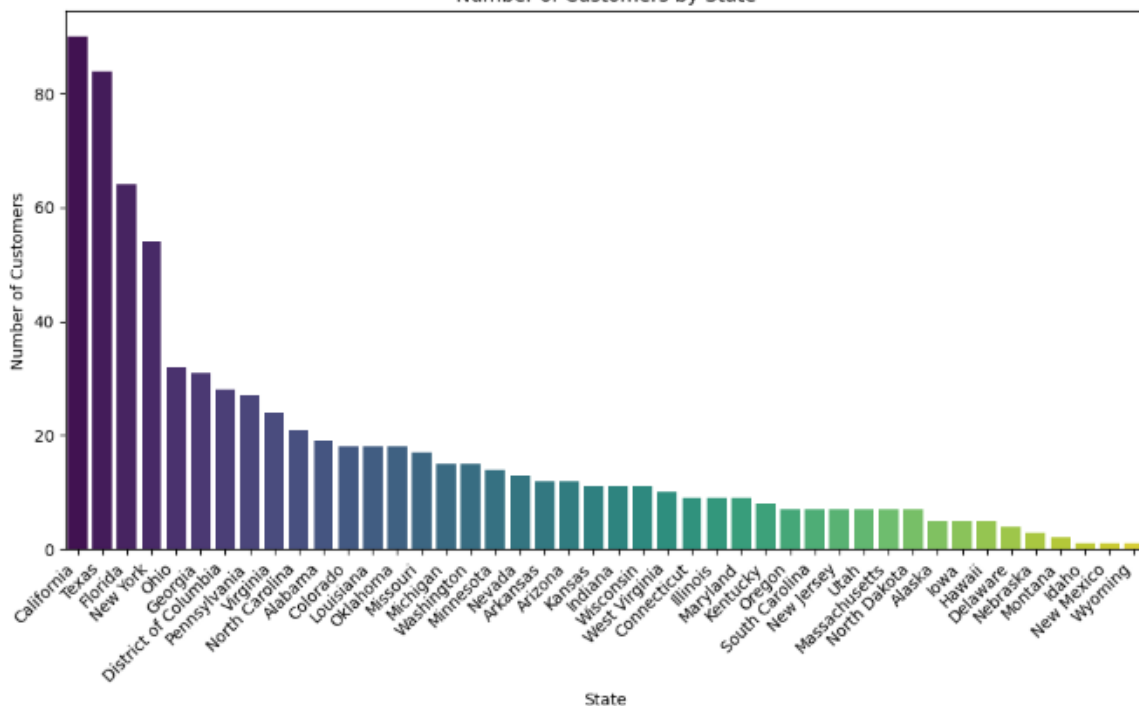
```

# Find the total quantity in each state
total_quantity = df_pre_processed_data.groupby('state_name')['Quantity'].sum()

# Plot the total quantity per state
total_quantity = total_quantity.sort_values(ascending=False)
plt.figure(figsize=(12, 6))
sns.barplot(x=total_quantity.index, y=total_quantity.values, palette='viridis')
plt.title('Total Quantity by State')
plt.xlabel('State')
plt.ylabel('Total Quantity')
plt.xticks(rotation=45, ha='right')
plt.show()

# Determine the state with the largest customer base
largest_total_quantity_state = total_quantity.idxmax()
print(f"The state with the most sales is: {largest_total_quantity_state}")

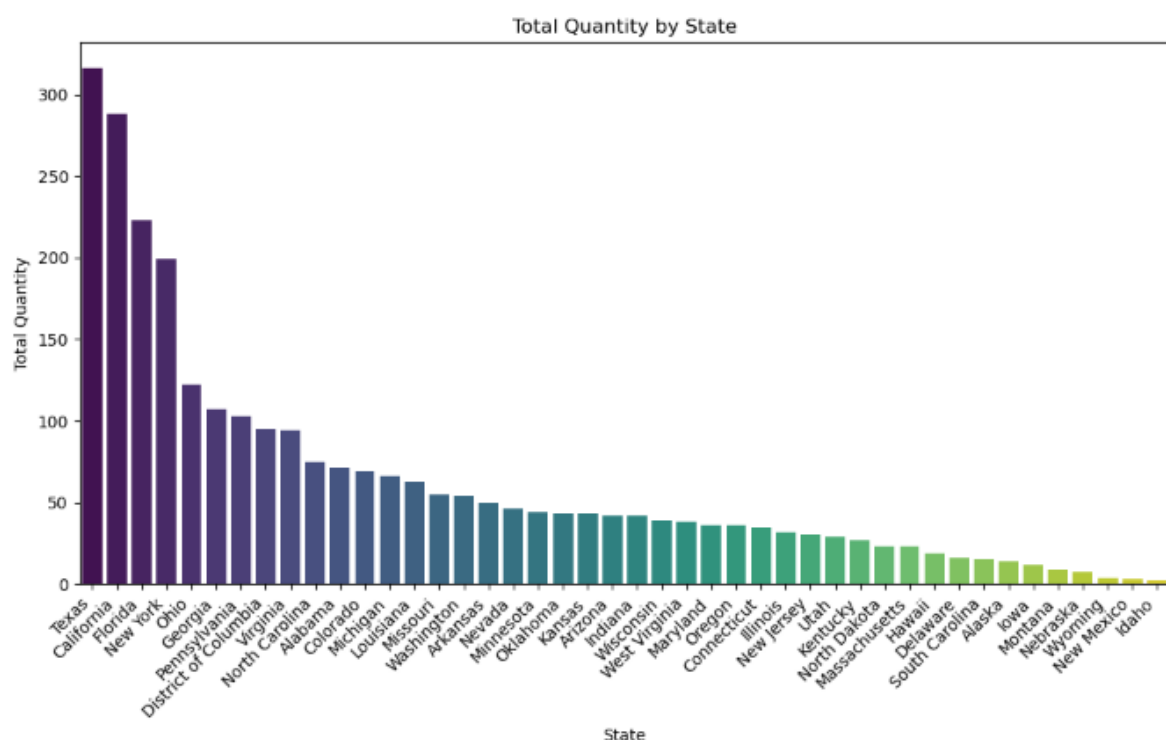
```



```
# Count the number of customers in each state
customer_counts = df_pre_processed_data['state_name'].value_counts()

# Plot the number of customers per state
plt.figure(figsize=(12, 6))
sns.barplot(x=customer_counts.index, y=customer_counts.values, palette='viridis')
plt.title('Number of Customers by State')
plt.xlabel('State')
plt.ylabel('Number of Customers')
plt.xticks(rotation=45, ha='right')
plt.show()

# Determine the state with the largest customer base
largest_customer_state = customer_counts.idxmax()
print(f"The state with the largest customer base is: {largest_customer_state}")
```



An observation here is that although Texas has the highest quantity, it was not the state with most customers. Instead, California which was second in customers had the highest quantity. We theorized that California bought less products on average, and it could be due to personal preference. Therefore, the seller should focus on Texas more by doing promotions because there is a high chance that the market there is yet to be fully saturated and people there tend to buy higher volumes of products. For the other states, the seller should consider to open more divisions to increase customers and sales.

4b) What coffee type/roast type/size does each country prefer?

```
# List of specific countries
selected_countries = ['United States', 'Ireland', 'United Kingdom']

# Create a custom color palette
mycolors = ["#FAE48A", "hotpink", "skyblue", "#11dc15"]

# Create subplots for each preference for each country
fig, axes = plt.subplots(nrows=len(selected_countries), ncols=3, figsize=(18, 6 * len(selected_countries)))

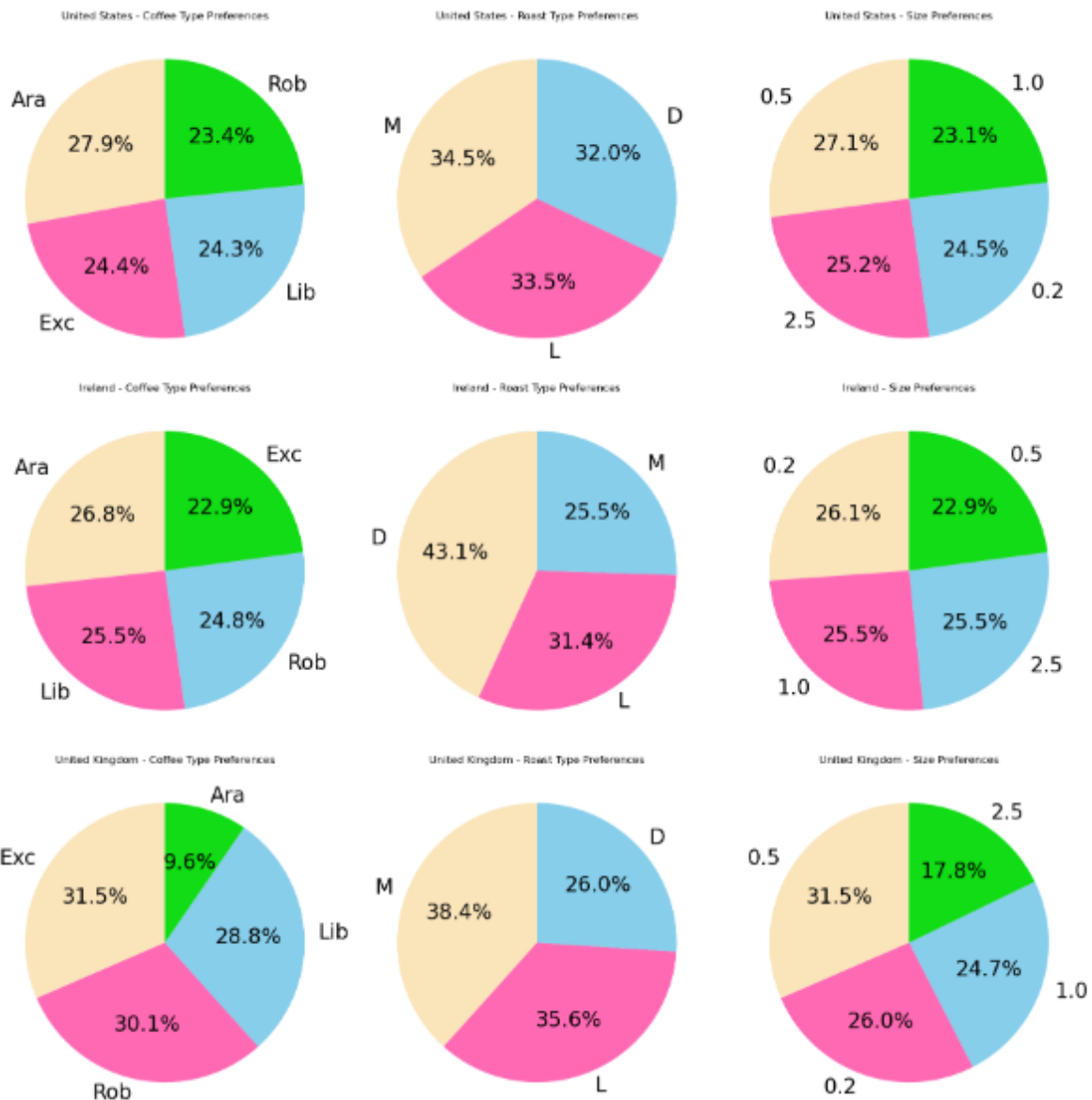
for i, country in enumerate(selected_countries):
    # Extract relevant columns for the current country
    country_data = df_pre_processed_data[df_pre_processed_data['Country'] == country][['Coffee Type', 'Roast Type', 'Size']]

    # Plot coffee type preferences
    coffee_type_distribution = country_data['Coffee Type'].value_counts()
    axes[i, 0].pie(coffee_type_distribution, labels=coffee_type_distribution.index, autopct='%1.1f%%', textprops={'fontsize': 24}, startangle=90, colors=mycolors)
    axes[i, 0].set_title(f'{country} - Coffee Type Preferences')

    # Plot roast type preferences
    roast_type_distribution = country_data['Roast Type'].value_counts()
    axes[i, 1].pie(roast_type_distribution, labels=roast_type_distribution.index, autopct='%1.1f%%', textprops={'fontsize': 24}, startangle=90, colors=mycolors)
    axes[i, 1].set_title(f'{country} - Roast Type Preferences')

    # Plot size preferences
    size_distribution = country_data['Size'].value_counts()
    axes[i, 2].pie(size_distribution, labels=size_distribution.index, autopct='%1.1f%%', textprops={'fontsize': 24}, startangle=90, colors=mycolors)
    axes[i, 2].set_title(f'{country} - Size Preferences')

# Adjust Layout
plt.tight_layout()
plt.show()
```



We will list the ones with greater difference:

For Coffee Type: It seems that Arabica coffee type is not popular in the United Kingdom.

For Roast Type: It seems that Dark roast type is very popular in Ireland.

For Size: It seems that 0.5 is quite popular in United Kingdom.

5) Loyalty card

5a) How many customers have loyalty card and how many don't?

5b) Do customers with loyalty card buy more?

```
# 5a) How many customers have Loyalty card and how many don't?
# Extract relevant column
loyalty_card_distribution = df_pre_processed_data['Loyalty Card'].value_counts()

# Plot a pie chart
plt.figure(figsize=(8, 8))
plt.pie(loyalty_card_distribution, labels=loyalty_card_distribution.index, autopct='%1.1f%%', textprops={'fontsize': 24}, colors=['skyblue', 'lightcoral'])
plt.title('Distribution of Customers with Loyalty Card')
plt.show()
```

```
# 5b) Do customers with loyalty card buy more?
# Aggregate data to get total quantity for having Loyalty card or not
agg_df_4 = df_pre_processed_data.groupby('Loyalty Card')['Quantity'].sum().reset_index()

# Rename a single column
agg_df_4.rename(columns={'Quantity': 'Total Quantity'}, inplace=True)

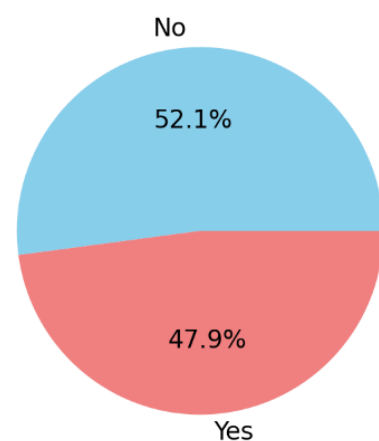
display(agg_df_4)
```

```
# Create a bar plot
plt.figure(figsize=(8, 6))
sns.barplot(x='Loyalty Card', y='Total Quantity', data=agg_df_4, palette='Set2')
plt.title('Total Quantity Purchased by Loyalty Card')
plt.xlabel('Loyalty Card')
plt.ylabel('Total Quantity')
plt.show()
```



	Loyalty Card	Total Quantity
0	No	1886
1	Yes	1665

Distribution of Customers with Loyalty Card



The customers with no loyalty card have a slight majority over those who own one. Unfortunately, there does not seem to be a significant difference between the total quantity purchased, it is also the customers having no loyalty card with more quantity purchased. This could mean that the perks of owning a loyalty card are not good enough to attract customers to buy and spend more on the products.

```
# Hypothesis Testing (e.g., t-test)
from scipy.stats import ttest_ind

loyalty_yes = df_pre_processed_data[df_pre_processed_data['Loyalty Card'] == 'Yes']['Quantity']
loyalty_no = df_pre_processed_data[df_pre_processed_data['Loyalty Card'] == 'No']['Quantity']

t_stat, p_value = ttest_ind(loyalty_yes, loyalty_no)
print(f"T-statistic: {t_stat}, P-value: {p_value}")
```

T-statistic: -1.3529501735561673, P-value: 0.1763779553140092

Given the p-value of 0.17637, which is greater than the significance level of 0.05, there is insufficient evidence to reject the null hypothesis. Therefore, we do not conclude that there is a significant difference in mean purchase quantities between customers with and without a loyalty card.

4.0 Conclusion

We have managed to gain insights by following the objectives we set.

1) Find the correlation of unit price, price per 100g, profit, and quantity.

1a) Is a higher unit price equivalent to more overall profit? Or will a high unit price affect the quantity bought and lower the overall profit?

Answer/Insight: In this dataset, the answer is yes. By raising the unit price, there will be more overall profit without having to worry about the reduction in quantities sold. A possible strategy is to raise the unit prices moderately.

1b) What is the unit price range that has a high quantity of sales?

Answer/Insight: The unit price range between 0-15 has the highest quantity of sales. Interestingly enough, the second highest quantity of sales is not the unit price range 15-20. Instead, it is followed by the price range 20-40, and the price range 15-20 has the lowest quantity of sales. A possible explanation is that people either want to save money if they are poor or spend more if they are rich.

2) Finding anomalies in quantity sales. If there are none, we can compare the sales

2a) Analysing profit mean and total sales

Answer/Insight: Liberica has the highest profit mean, which means by selling the same quantity of different coffee bean products, Liberica brings in the most profit. As for the total sales, it shows that Ara has a slight advantage in total quantity sold compared to other coffee types.

2b) Is the most profitable coffee type also the most sold coffee type?

Answer/Insight: Liberica coffee beans has the lowest demand (lowest quantity ordered), yet the total profit it brings is the lowest. On the other hand, Arabica coffee beans has the highest demand (highest quantity ordered), but its total profit is ranked second lowest. It can be concluded that the seller should put more focus into promoting Liberica coffee beans as it brings the most profit for them.

3) Time analysis of total sales

3a) Total sales and sales trend each year

Answer/Insight: The trend is always increasing at the start of the year, then it plummets around April to May. It is a recurring pattern for all three years. This could be due to the expiry date (or "Best if Used before" date) of the coffee bean product being counted in years. This could explain the certain spike in sales every year at the same month. Most people probably bought their products at the start of the year initially, so naturally the product will be finished up or expired by the start of a few years later too. Therefore, customers will buy the products at the same month of the year again.

3b) Prediction of sales (quantity)

Answer/Insight: The prediction is that average sales will be around 80 for the year 2022. An advanced prediction involving fluctuations is not possible as there is no clear relationship between the factors that affect the sales.

3c) What coffee type is trending each year?

Answer/Insight: Liberica is not a popular coffee bean choice. On the other hand, Robusta coffee bean seems to be the overall popular coffee bean choice.

4) Analyse country sales on coffee bean

4a) How many orders and sales from each country? If there is a country with high sales/orders, do more detailed analysis/analytics on it

Answer/Insight: United States contributes most of the orders. We theorized that this could be due to the lack of franchisees in other nations. We suggested that the owner/seller could open more divisions to reach more customers. Additionally, we decided to include the analysis for the states in United States as it had the most sales.

4b) What coffee type/roast type/size does each country prefer?

Answer/Insight:

For Coffee Type: It seems that Arabica coffee type is not popular in the United Kingdom.

For Roast Type: It seems that Dark roast type is very popular in Ireland.

For Size: It seems that 0.5 is quite popular in United Kingdom.

5) Loyalty card

5a) How many customers have loyalty card and how many don't?

Answer/Insight: There is a slight majority of customers who don't have loyalty card compared to those who have one.

5b) Do customers with loyalty card buy more?

Answer/Insight: There does not seem to be a significant difference between the total quantity purchased. A possible explanation is the perks of owning a loyalty card are not good enough to attract customers to buy and spend more on the products. The seller should focus on giving out better perks to those who own loyalty cards so the customer who have the loyalty cards will buy and spend more.

APPENDIX 1

MARKING RUBRICS

Component Title	Assignment					Percentage (%)	100
Criteria	Score and Descriptors					Weight (%)	Marks
	Excellent (5)	Good (4)	Average (3)	Need Improvement (2)	Poor (1)		
Data Collection	Data collected and exceptionally relevant to assignment	Data collected and relevant to assignment	Data collected but limited relevant to assignment	Data collected but mostly irrelevant to the assignment	Data collected but irrelevant to the assignment	15	
Data Preprocessing	Excellent data preprocessing conducted	Appropriate data preprocessing conducted	Limited data preprocessing conducted	Some of the data preprocessing conducted	No data preprocessing conducted	20	
Data Analysis and Data Analytics	Excellent data analysis and analytic conducted	Appropriate data analysis and analytic conducted	Limited data analysis and analytic conducted	Some of the data analysis and analytic conducted	No data analysis and analytic conducted	20	
Data visualization	Excellent data visualization conducted	Appropriate data visualization conducted	Limited data visualization conducted	Some of the data visualization conducted	No data visualization conducted	20	
Conclusion	Engaging and informative conclusion provided, with comprehensive and insightful summarization of key findings.	Clear and concise conclusion provided, with relevant summarization of key findings.	Adequate conclusion provided, but lacking clear summarization of key findings	Vague or insufficient conclusion provided	No conclusion provided or completely irrelevant	15	
Delivery (individual)	Exceptionally coherent, engaging, and confident, with no distracting factors.	Coherent, engaging, and confident with only minor issues.	Adequately coherent, engaging, and few distracting factors.	Somewhat coherent, partially engaging, some distracting factors.	Incoherent, difficult to understand, lacks engagement, and distracting.	5	

Teamwork (individual)	Exceptional teamwork demonstrated , highly collaborative , and consistent participation with no areas for improvement .	Effective teamwork demonstrated ,collaborative,and consistent participation with minor improvements needed.	Adequate teamwork demonstrated, cooperative, and consistent participation.	Minimal teamwork demonstrated ,unenthusiastic,and inconsistent participation.	No teamwork demonstrate, uncooperative,and disruptive.	5	
TOTAL						100	

Note to students: Please include the marking rubric when submitting your coursework.